Object Oriented Design and Programming

Dr. Sanjeev Patel

Asst. Professor Grade-I

Department of Computer Science and Engineering

National Institute of Technology Rourkela, Odisha

Outline

- Introduction to Design Patterns
- Design Patterns and Its Characteristics
- Design Patterns vs Algorithms
- Types of Design Patterns
- Singleton Design Pattern
- Factory Design Pattern
- UML
- Usecase Diagram
- Class Diagram
- Sequence Diagram

Introduction to Design Patterns

- Design patterns provide proven solutions
- A design pattern is a high-level description of a solution to a common design problem,
  - which can be adapted to different programming situations.
- Enhance scalability and flexibility
- Widely used in modern software development
- Design patterns are standardized approaches to solving recurring design problems, improving code reusability and efficiency.

Design Patterns and Its Characteristics

- What are Design Patterns?
  - Design patterns are typical solutions to common software design problems
  - Act as reusable blueprints
  - Help solve recurring design issues
- Key Characteristics
  - Not ready-made code or functions
  - Provide a general solution concept
  - Can be customized based on application needs
  - Improve code reusability and maintainability

Design Patterns vs Algorithms

➢Design Patterns

- Patterns are not directly copy-paste solutions
- Require implementation based on context
- Same pattern can have different code implementations in different programs
- High-level conceptual solutions
- Flexible implementation

➢Algorithms

- Step-by-step clear instructions
- Fixed procedure
- An analogy to an algorithm is a cooking recipe: both have clear steps to achieve a goal.
- On the other hand, a pattern is more like a blueprint:

Types of Design Patterns

- Types of Design Patterns
  - Creational → Object creation
  - Structural → Object structure
  - Behavioral → Object interaction
- Singleton Method- This ensure that a class has only one instance, while providing a global access point to this instance.
- Factory Method- Factory Method is a creational design pattern. It provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.
- Prototype Method-It is a creational design pattern that lets you copy existing objects without making your code dependent on their classes.
- Observer Method-The Observer pattern is a behavioral design pattern used to establish a one-to-many dependency between objects.

Singleton Design Pattern

•The Singleton pattern solves two problems at the same time, violating the Single Responsibility Principle.

- **Ensure that a class has just a single instance**.
  - •Why would anyone want to control how many instances a class has?
  - •The most common reason for this is to control access to some shared resource—for example, a database or a file.
- **Provide a global access point to that instance**.
  - Remember those global variables that you (all right, me) used to store some essential objects?
  - While they're very handy, they're also very unsafe since any code can potentially overwrite the contents of those variables and crash the app.

Singleton Design Pattern: Implementation

- Add a private static field to the class for storing the singleton instance.
- Declare a public static creation method for getting the singleton instance.
- Implement “lazy initialization” inside the static method. It should create a new object on its first call and put it into the static field. The method should always return that instance on all subsequent calls.
- Make the constructor of the class private. The static method of the class will still be able to call the constructor, but not the other objects.
- Go over the client code and replace all direct calls to the singleton’s constructor with calls to its static creation method.

Factory Design Pattern

- **Factory Method** is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.
- Use the Factory Method when you want to save system resources by reusing existing objects instead of rebuilding them each time.
- Use the Factory Method when you don’t know beforehand the exact types and dependencies of the objects your code should work with.
- Use the Factory Method when you want to provide users of your library or framework with a way to extend its internal components.

Factory Design Pattern

- Make all products follow the same interface. This interface should declare methods that make sense in every product.
- Add an empty factory method inside the creator class. The return type of the method should match the common product interface.
- In the creator’s code find all references to product constructors. One by one, replace them with calls to the factory method, while extracting the product creation code into the factory method.
- Now, create a set of creator subclasses for each type of product listed in the factory method. Override the factory method in the subclasses and extract the appropriate bits of construction code from the base method.
- If there are too many product types and it doesn’t make sense to create subclasses for all of them, you can reuse the control parameter from the base class in subclasses.
- If, after all of the extractions, the base factory method has become empty, you can make it abstract. If there’s something left, you can make it a default behavior of the method.

A Use Case Driven Approach [6]

- The object-oriented software development life cycle(SDLC) consists of three macro processes:
  - Object-oriented analysis
  - Object-oriented design
  - Object-oriented implementation
- The use case model can be employed throughout most activities of software development.

Object-oriented Analysis [6]

- Scenarios are a great way of examining who does what in the interactions among objects and what role they play; i.e., their interrelationships.
- This intersection among object's roles to achieve a given goal is called collaboration.
- A use case is a typical interaction between a user and system that captures user's goals and needs.
- The use case model represents the user's view of the system or user's needs.

Object-oriented Design [6]

- Object-oriented design and object-oriented analysis are distinct disciplines, but they can be intertwined.
- Object-oriented development is highly incremental.
- If you start with object-oriented analysis, model it, create an object-oriented design
- Then do some more of each, again and again, gradually refining and completing models of the system.

Object-oriented Design [6]

- First, build the object model based on objects and their relationships, then iterate and refine the model:
  - Design and refine classes
  - Design and refine attributes
  - Design and refine methods
  - Design and refine structures
  - Design and refine associations

Object-oriented Design [6]

- Here are a few guidelines to use in your object-oriented design:
  - Reuse, rather than build, a new class. Know the existing classes.
  - Design a large number of simple classes, rather than a small number of complex classes.
  - Design methods.
  - Critique what you have proposed. If possible, go back and refine the classes.

Object Modelling: UML [1]

- UML is language for creating models and it is a modeling language.
- It is not a system design or development methodology
- We construct a model to manage the complexity in a problem
- Used to document object-oriented analysis and design results and independent of any specific design methodology.

Why are UML Models Required? [1]

- Modelling is an abstraction mechanism:
  - Capture only important aspects and ignores the rest.
  - Different models obtained when different aspects are ignored.
  - An effective mechanism to handle complexity.
- UML is a graphical modelling technique
- Easy to understand and construct

Views of a system [1]

- User model view
  - This view represents the system (product) from the user's (called "actors" in UML) perspective.
- Structural model view
  - Data and functionality is viewed from inside the system. That is, static structure (classes, objects, and relationships) is modeled.
  - It is also known as static model
- Behavioral model view
  - This part of the analysis model represents the dynamic or behavioral aspects of the system.
  - It is also known as dynamic model

Static Model vs Dynamic Model [3]

- Static Model
  - It describes the static structure of the system using object classes and their relationships.
  - It includes generalization relationships, uses/used-by relationships and composition relationships
- Dynamic Model
  - It describes the dynamic structure of the system and show the interactions between the system objects
  - Object interactions that includes the sequence of service requests made by objects

Use case Diagram: Identification of Use Cases[1]

1. Actor-based:

- Identify the actors related to a system or organization.
- For each actor, identify the processes they initiate or participate in.

2. Event-based

- Identify the external events that the system must respond to.
- Relate the events to actors and use cases.

Functional Analysis: Use Case Diagram

- Use cases, actors, systems, relationship
  - Use case can have relationship
    - include, and extend
- Modeling the behavior of an element
- Realizing use cases with collaborations

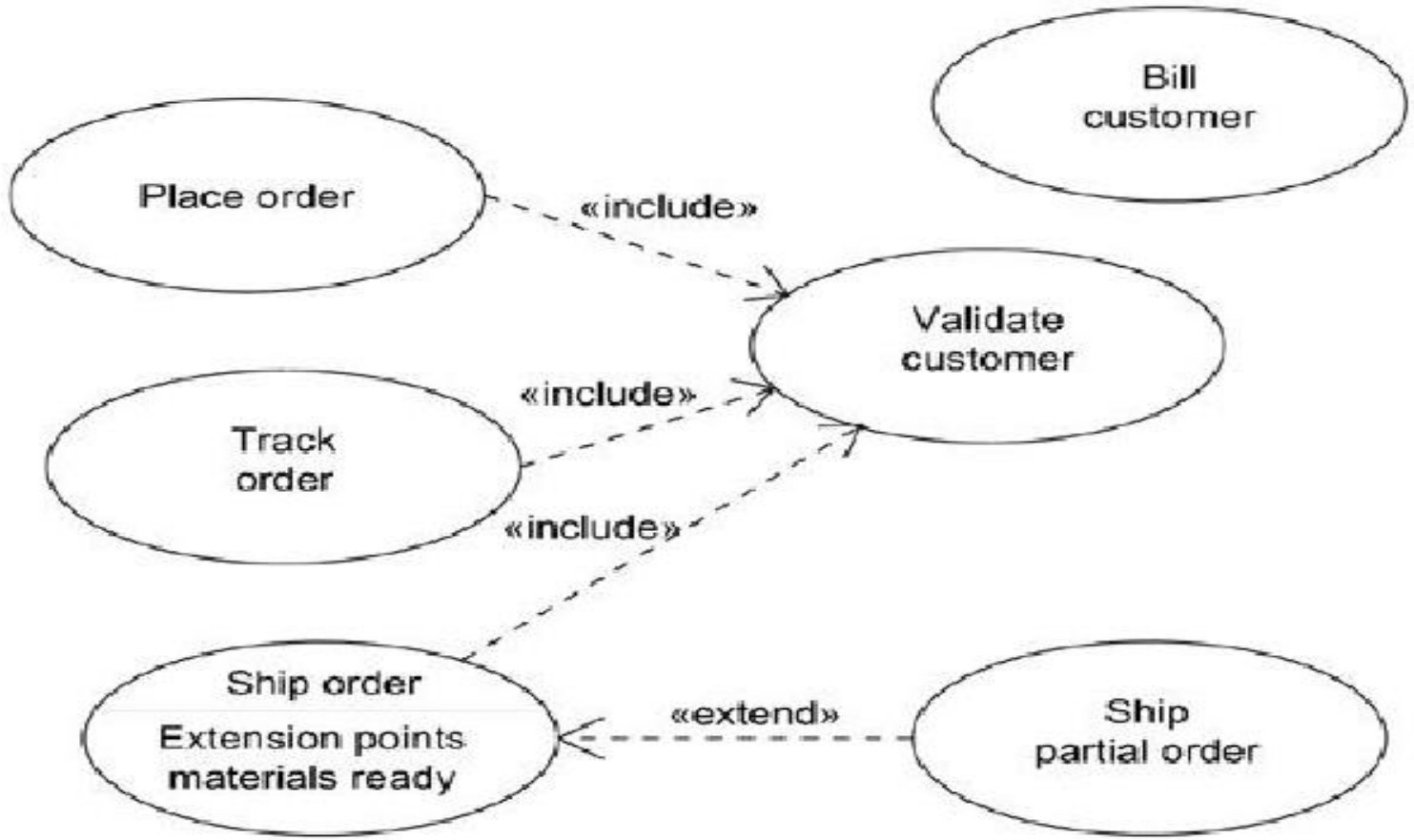
Elements of Use Case [7]

- Actor
  - An actor represents a role that an outsider takes on when interacting with the business system.
  - i.e. an actor can be a customer, a business partner, a supplier, or another business system.
- System or Subject
  - A subject describes a business system that has one or more business use cases attached to it.
  - A subject is represented by a rectangle that surrounds attached business use cases and is tagged with a name.

Elements of Use Case [7]

- Association or Relationship
  - An association is the relationship between an actor and a business use case.
  - It indicates that an actor can use a certain functionality of the business system or the business use case.
- Business Use Case
  - A business use case describes the interaction between an actor and a business system
  - It means, it describes the functionality of the business system that the actor utilizes.

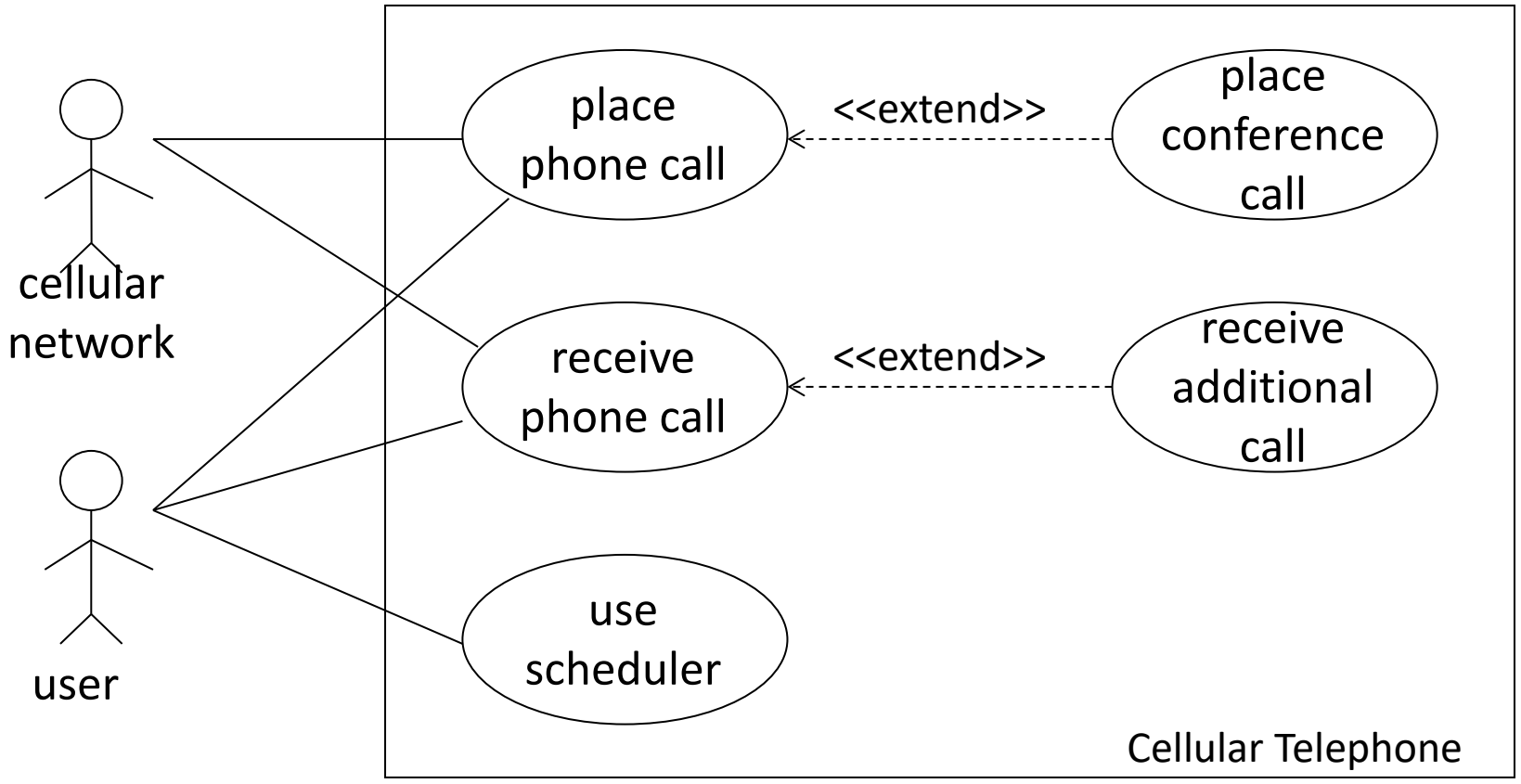
Include vs Extend



Constructing Use Case Diagrams [7]

- Collect information sources
  - How am I supposed to know that?
- Identify potential actors
  - Which partners and customers use the goods and services of the business system?
- Identify potential business use cases
  - Which goods and services can actors draw upon?
- Connect business use cases
  - Who can make use of what goods and services of the business system?
- Describe actors
  - Who or what do the actors represent?

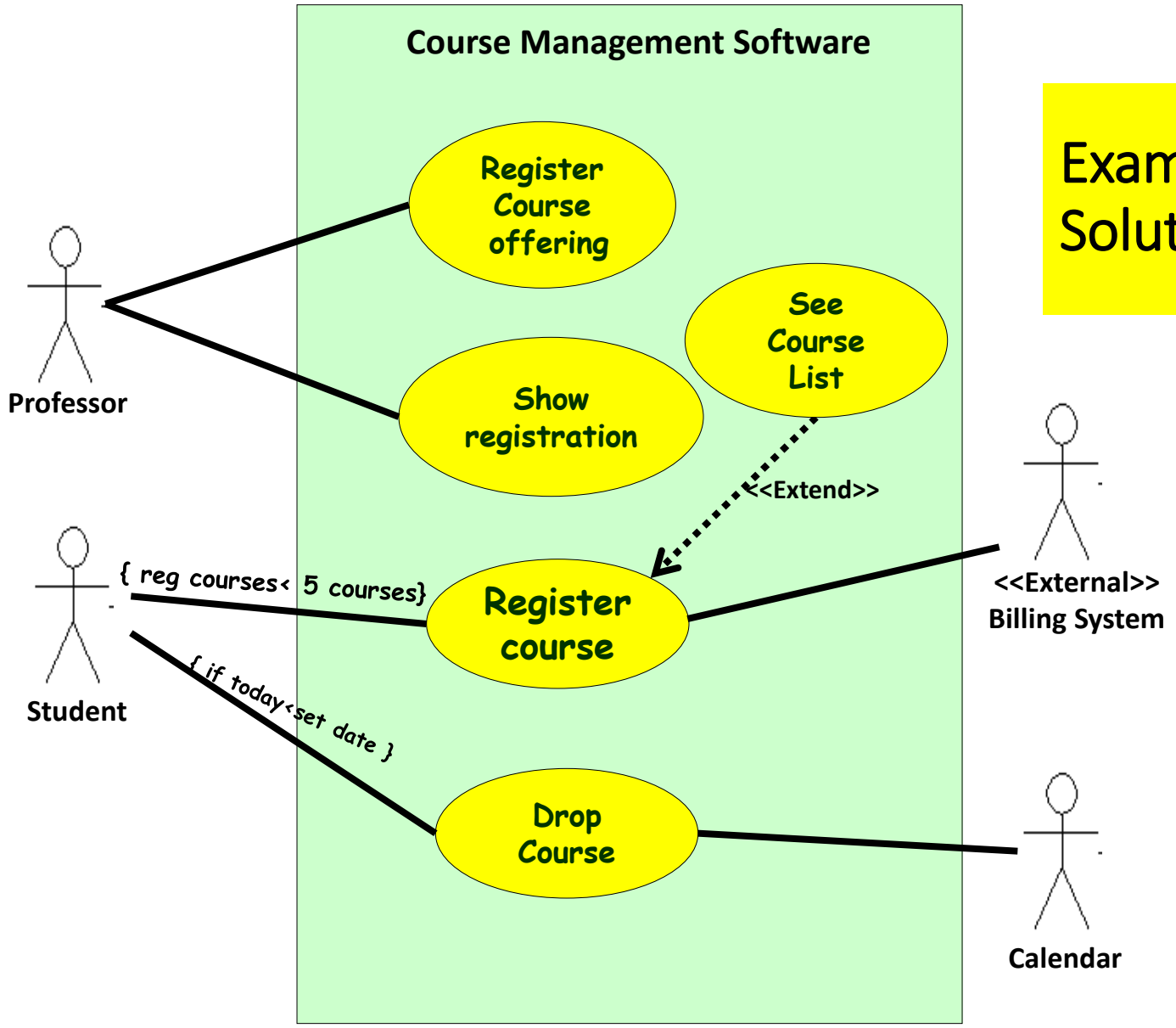
USE CASE Example

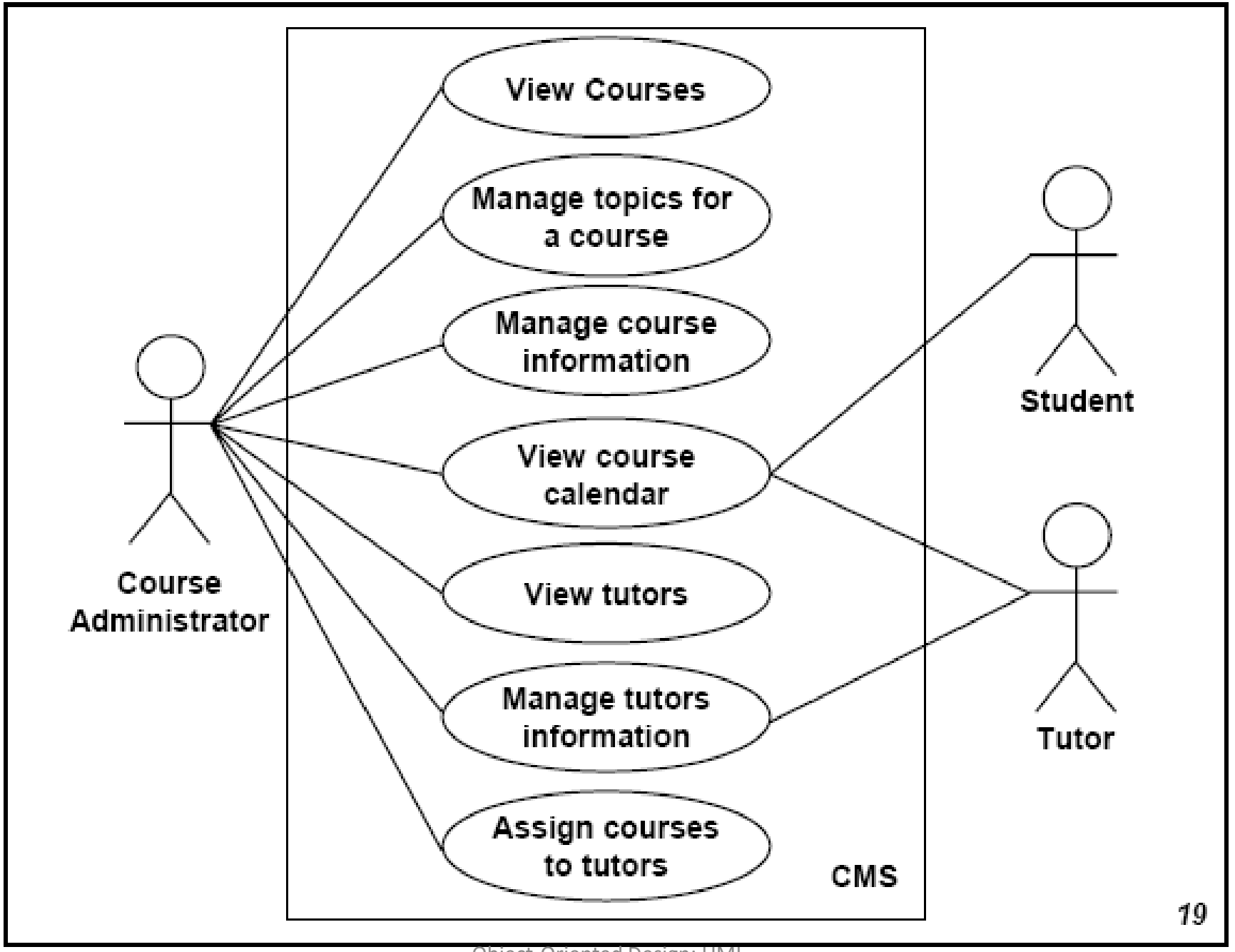


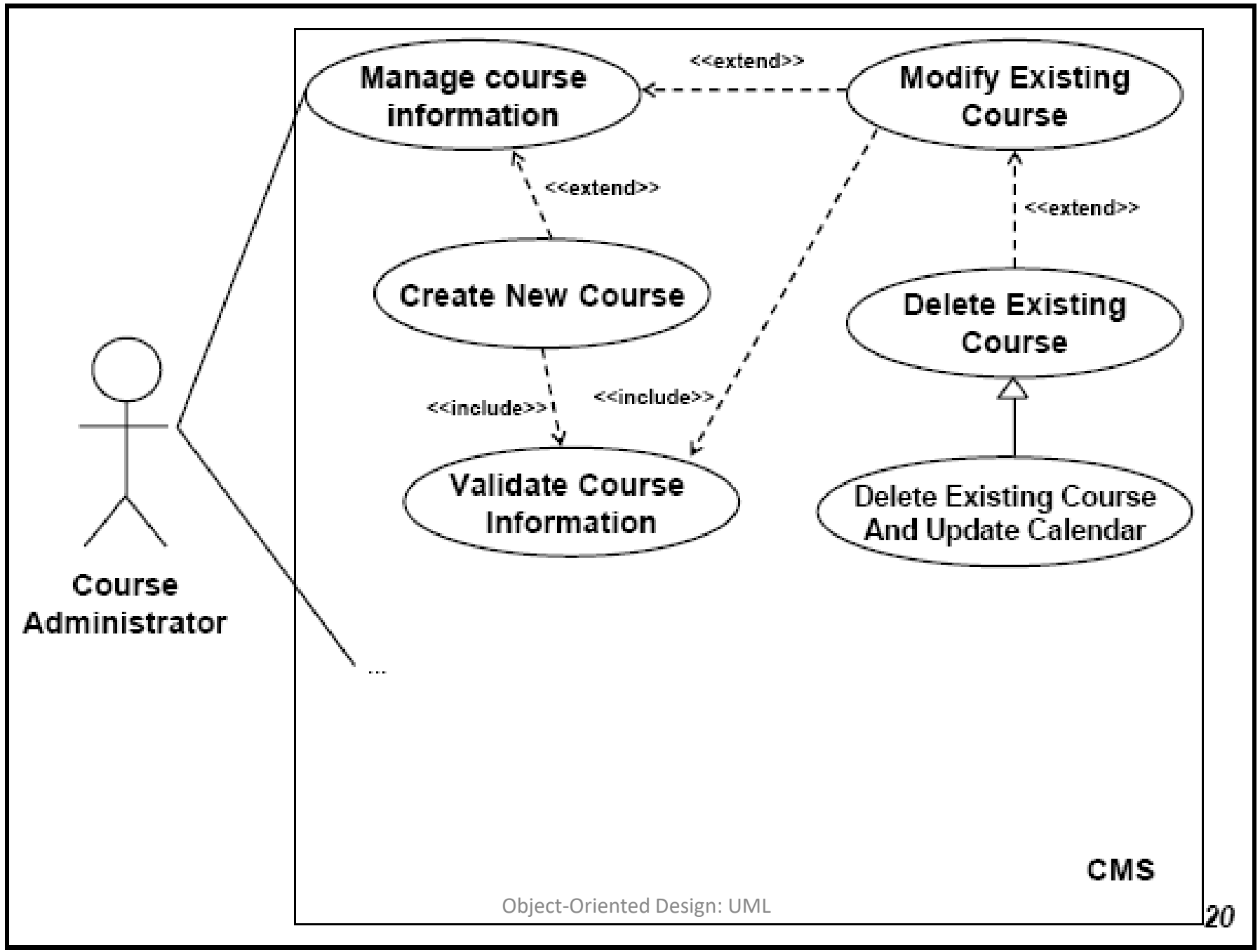
Example: Use Case Model for Course Management Software [1]

- At the beginning of each semester,
  - Each professor shall register the courses that he is going to teach.
- A student can select up to four-course offerings.
  - **During registration a student can request a course catalogue showing course offerings for the semester.**
  - **Information about each course such as professor, department and prerequisites would be displayed.**
  - **The registration system sends information to the billing system, so that the students can be billed for the semester.**
- For each semester, there is a period of time during which dropping of courses is permitted.
- Professors must be able to access the system to see which students signed up for each of their course offerings.

Example: Model Solution [1]




View Courses
Manage topics for a course
Manage course information
View course calendar
View tutors
Manage tutors information
Assign courses to tutors
CMS
Course Administrator
Student
Tutor
19


Manage course information
<<extend>>
Modify Existing Course
<<extend>>
<<extend>>
Create New Course
Delete Existing Course
<<include>>
<<include>>
Validate Course Information
Delete Existing Course And Update Calendar
Course Administrator
...
CMS
20

Classes

- A class is a collection of objects with common structure, common behavior, common relationships and common semantics
- Classes are found by examining the objects in sequence and collaboration diagram
- A class is drawn as a rectangle with three compartments
- Classes should be named using the vocabulary of the domain
  - Naming standards should be created
  - e.g., all classes are singular nouns starting with a capital letter

Class Diagrams [1]

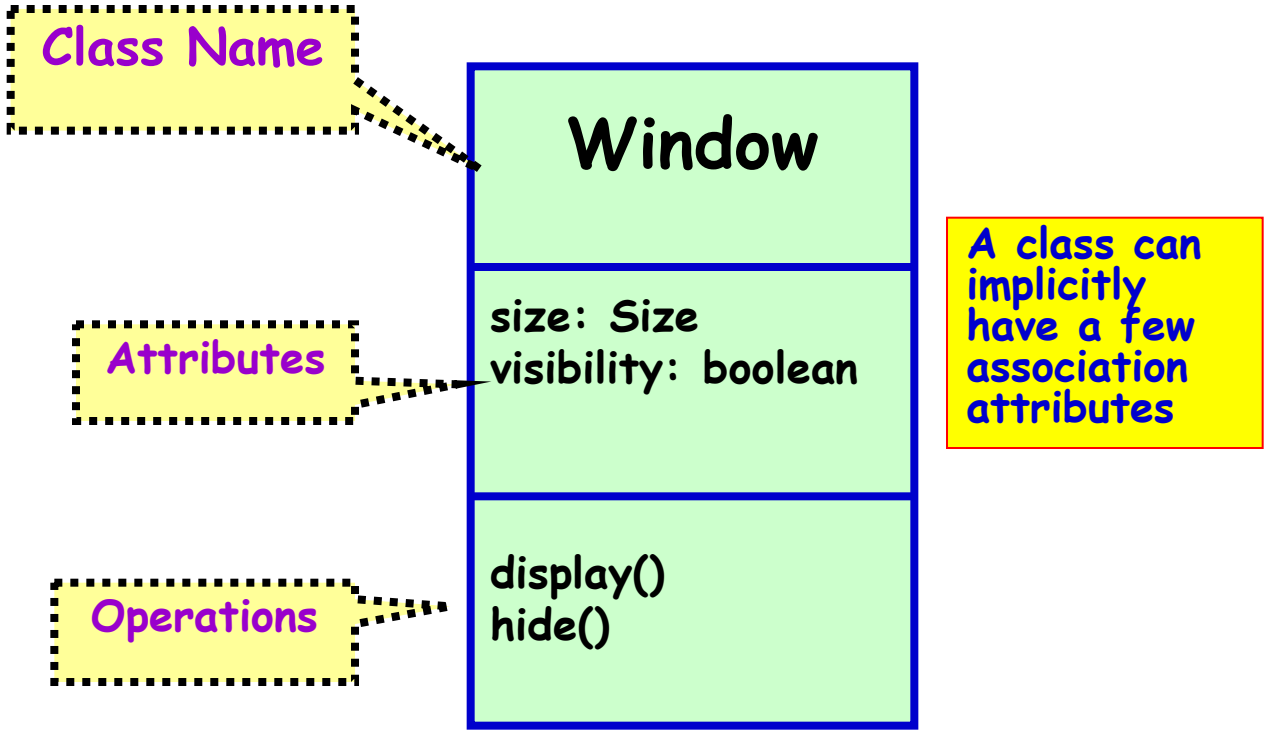
- A class diagram shows the existence of classes and their relationships in the logical view of a system
  - Entities with common features, i.e. attributes and operations.
  - Represented as solid outline rectangle with compartments.
  - Compartments for **name, attributes, and operations.**
  - Attribute and operation compartments are optional depending on the purpose of a diagram.
- UML modeling elements in class diagrams
  - Classes and their structure and behavior
  - Association, aggregation, dependency, and inheritance relationships
  - Multiplicity and navigation indicators
  - Role names

Class Based Modeling

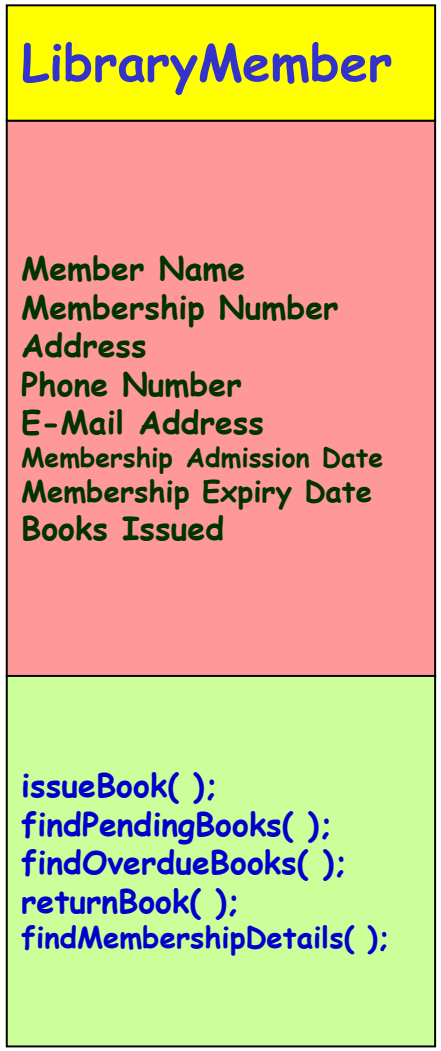
- Identify analysis classes by examining the problem statement
- Use a “grammatical parse” to isolate potential classes
- Identify the attributes of each class
- Identify operations that manipulate the attributes

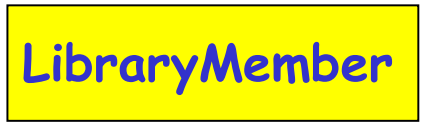
UML Class Representation [1]

- A class represents a set of objects having similar attributes, operations, relationships and behavior.



Different representations of the LibraryMember class[1]





Example UML Classes

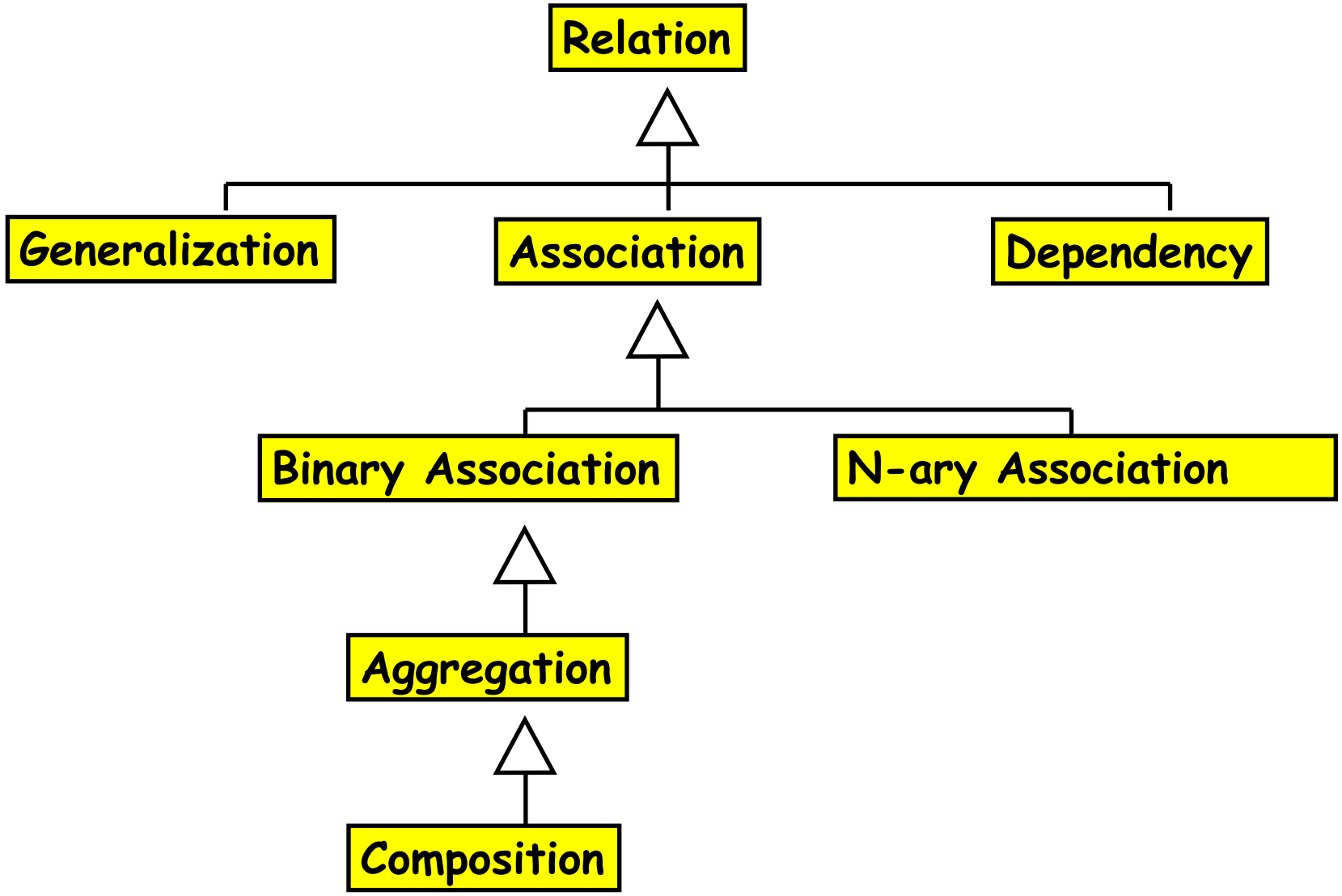
Relationships

- A *relationship is a connection among things.*
- In object-oriented modeling, the following important relationships are
  - Dependencies
  - Generalizations
  - Associations
- Other relationships are
  - Containment- Aggregation and Composition
  - Persistence

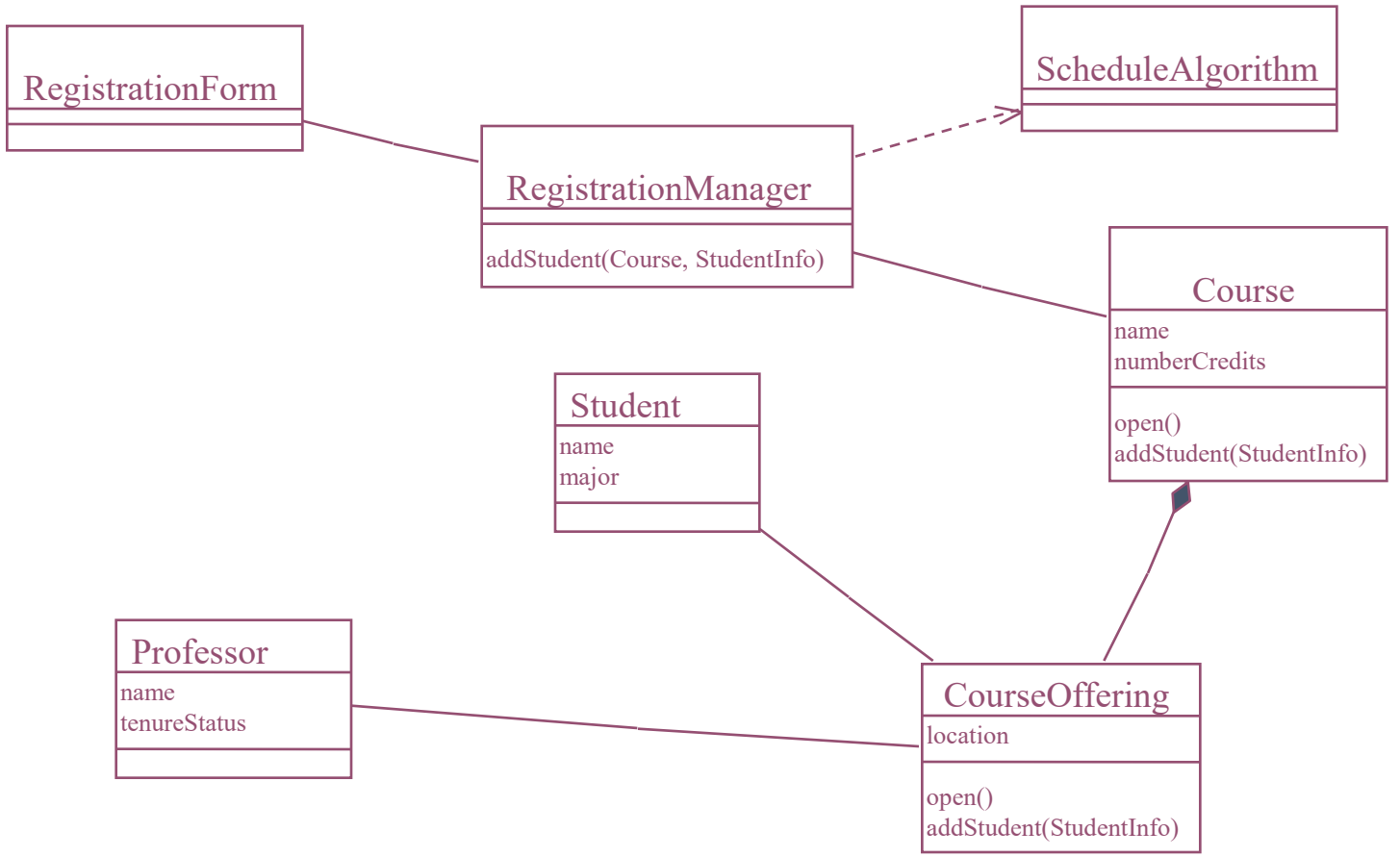
Relationships

- An association is a bi-directional connection between classes
  - An association is shown as a line connecting the related classes
- An aggregation is a stronger form of relationship where the relationship is between a whole and its parts
  - An aggregation is shown as a line connecting the related classes with a diamond next to the class representing the whole
- A dependency relationship is a weaker form of relationship showing a relationship between a client and a supplier where the client does not have semantic knowledge of the supplier
- A dependency is shown as a dashed line pointing from the client to the supplier

Types of Class Relationships[1]



Relationships

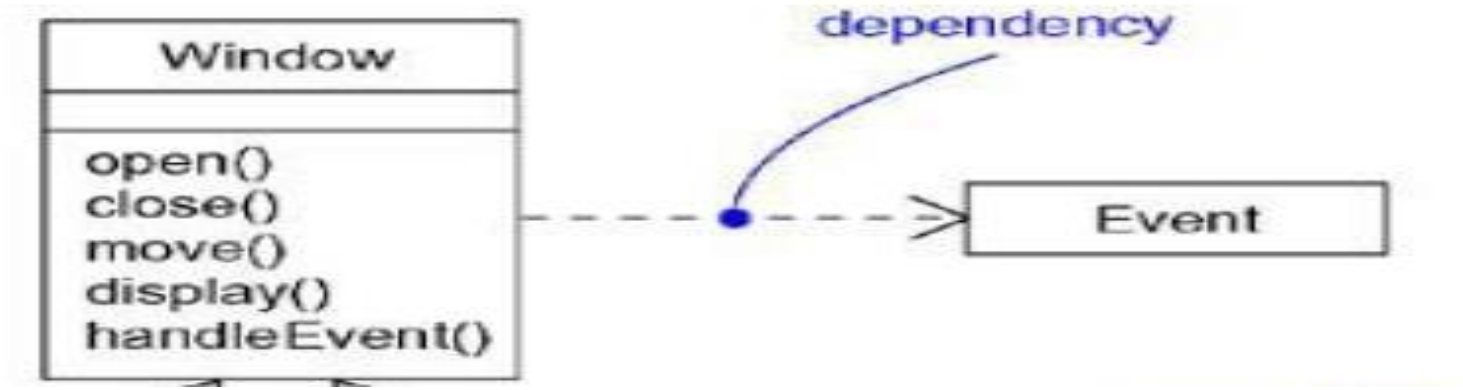


Associations vs Dependencies

- Two analysis classes are often related to one another in some fashion
  - In UML these relationships are called *associations*
  - Associations can be refined by indicating *multiplicity* (the term *cardinality* is used in data modeling
- In many instances, a client-server relationship exists between two analysis classes.
  - In such cases, a client-class depends on the server-class in some way and a *dependency relationship* is established

Dependency

- A *dependency is a using relationship that states that a change in specification of one thing*.
- for example, class Event may affect another thing that uses it (for example, class Window), but not necessarily the reverse.
- Graphically, a dependency is rendered as a dashed directed line, directed to the thing being depended on.

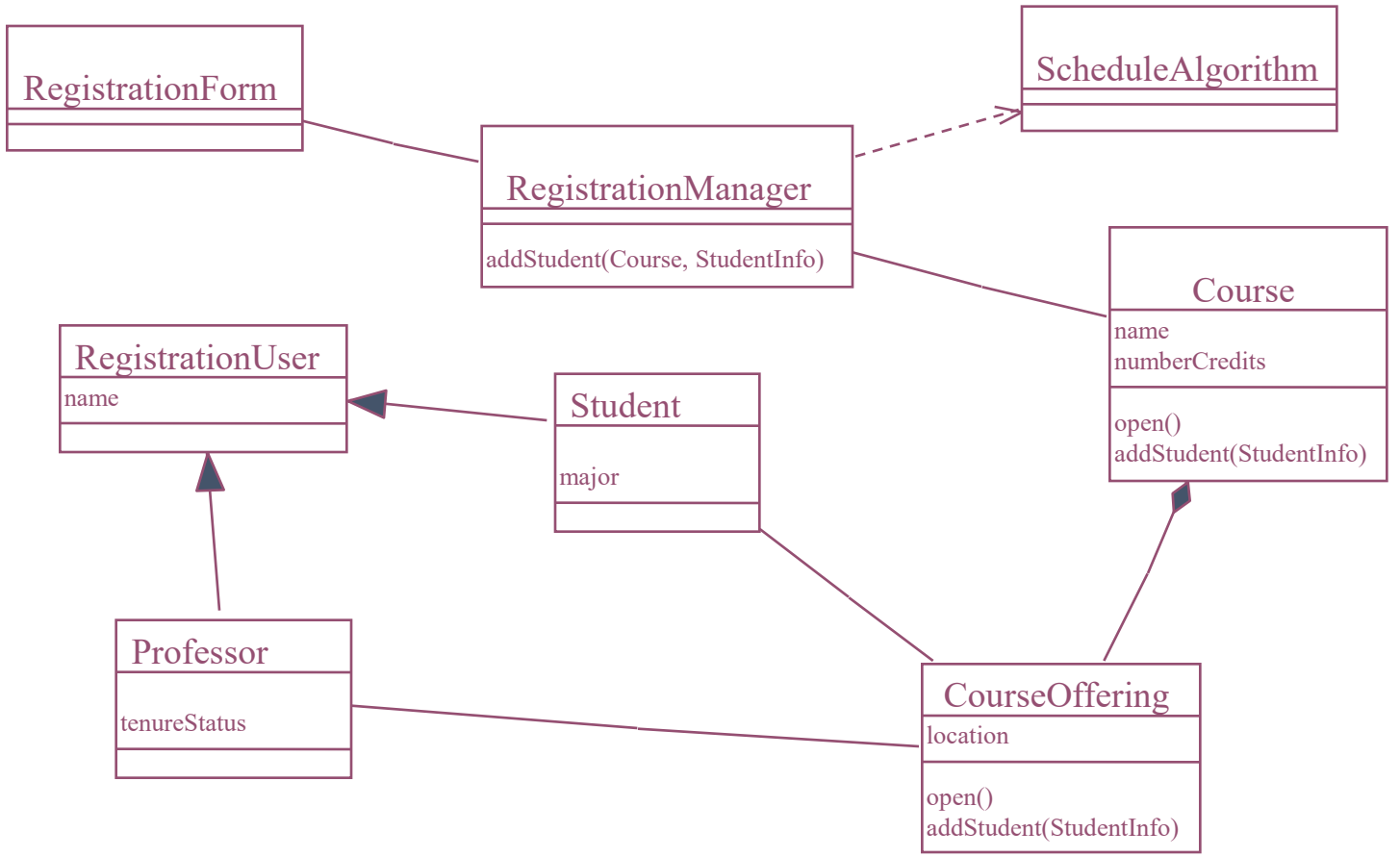


Class Dependency

Inheritance

- Inheritance is a relationships between a superclass and its subclasses
- There are two ways to find inheritance:
  - Generalization
  - Specialization
- Common attributes, operations, and/or relationships are shown at the highest applicable level in the hierarchy

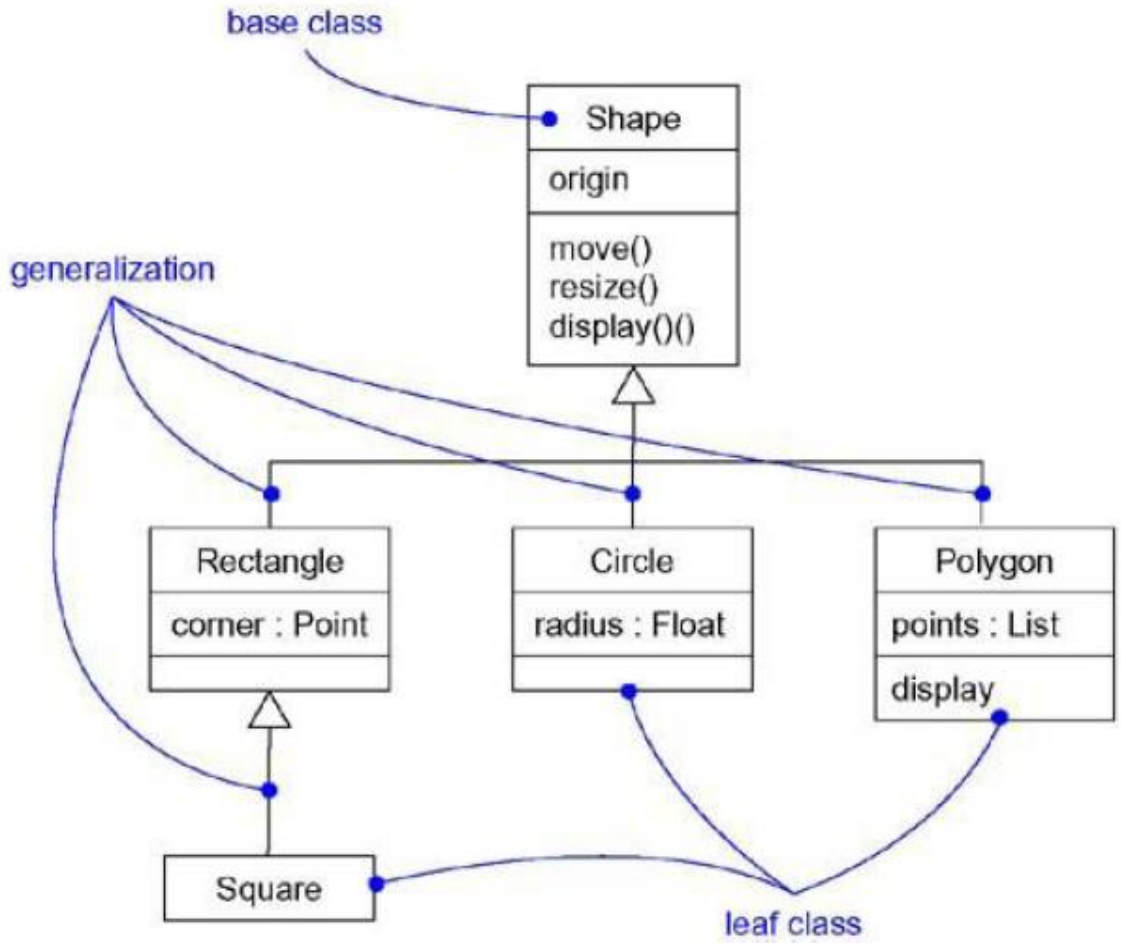
Inheritance



Generalizations

- *Generalizations connect generalized classes to more-specialized ones in what is known as* subclass/superclass or child/parent relationships.
- For example,
  - a bay window is a kind of window with large, fixed panes;
  - a patio window is a kind of window with panes that open side to side.
- A child inherits the properties of its parents, especially their attributes and operations.
- Graphically, generalization is rendered as a solid directed line with a large open arrowhead, pointing to the parent

Generalizations



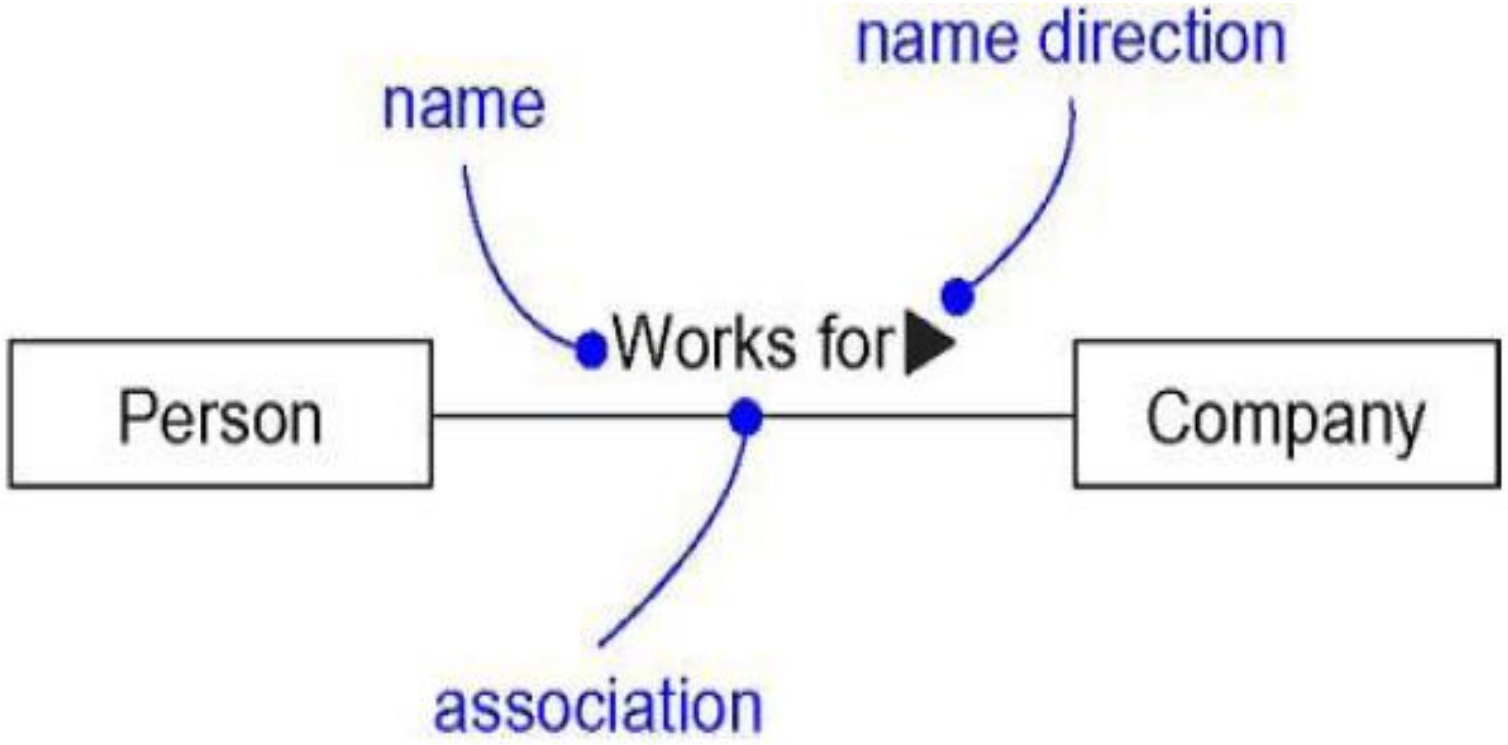
Generalizations

- A class may have zero, one, or more parents.
- A class that has no parents and one or more children is called a root class or a base class.
- A class that has no children is called a leaf class.
- A class that has exactly one parent is said to use single inheritance.
- A class with more than one parent is said to use multiple inheritance.

Association

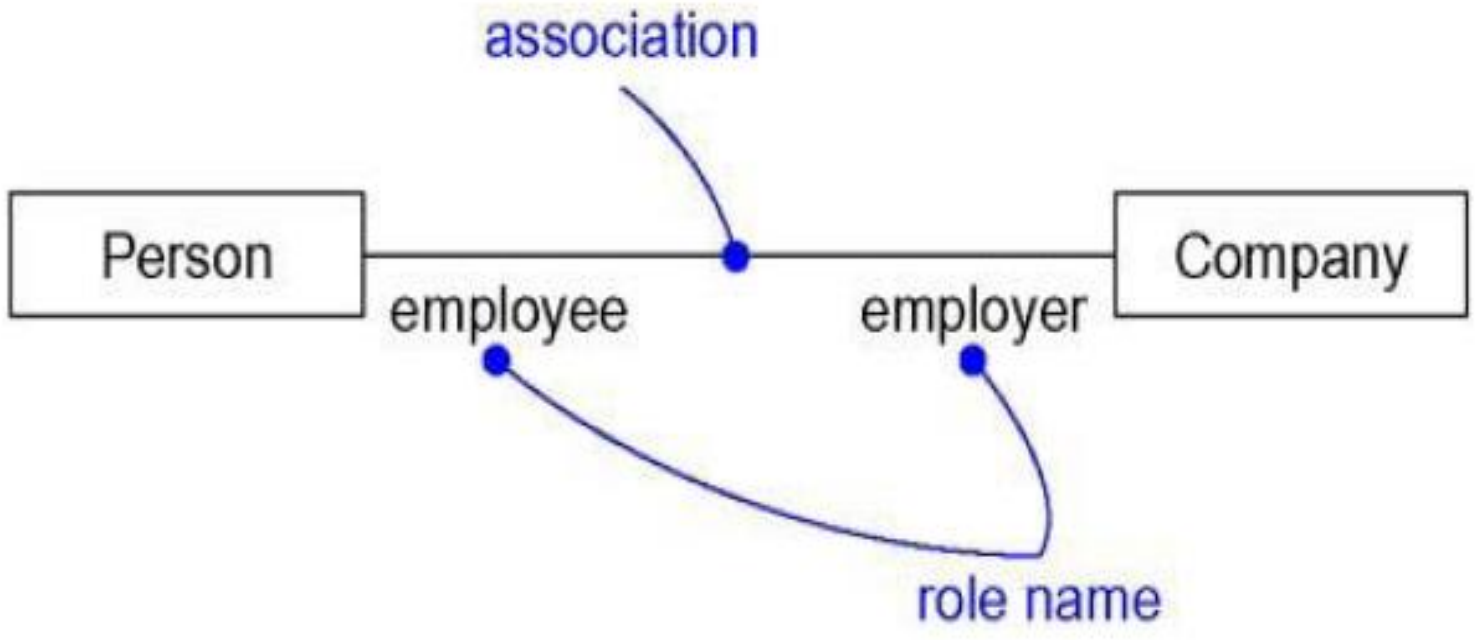
- An *association is a structural relationship that specifies that objects of one thing are connected to* objects of another.
- An association can have a name, and you use that name to describe the nature of the relationship. So that there is no ambiguity about its meaning.
- you can give a direction to the name by providing a direction triangle that points in the direction you intend to read the name.

Association

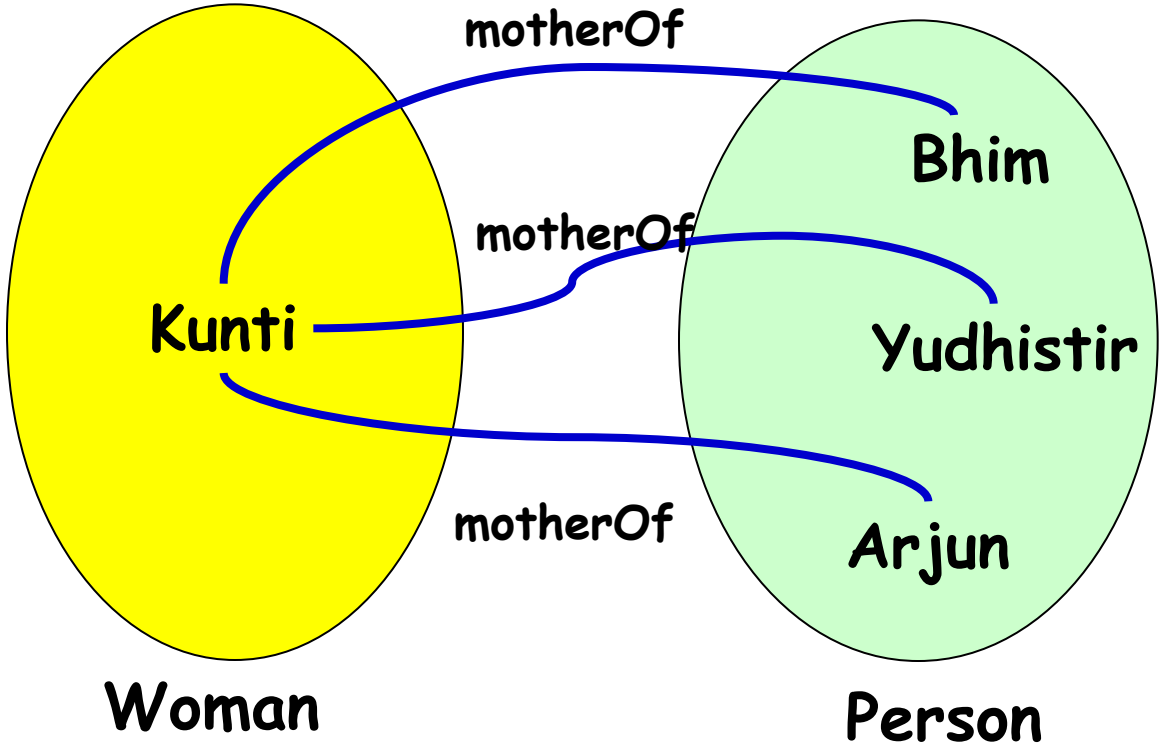


Association

- When a class participates in an association, it has a specific role that it plays in that relationship; a role is just the face the class at the near end of the association presents to the class at the other end of the association.

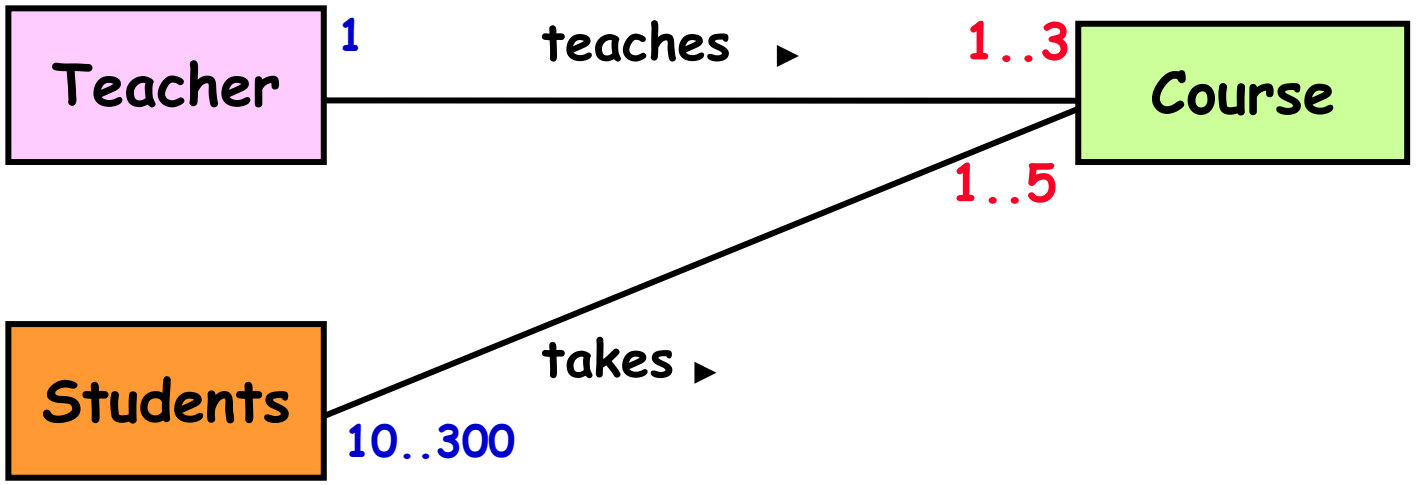


Multiple Association - example[1]



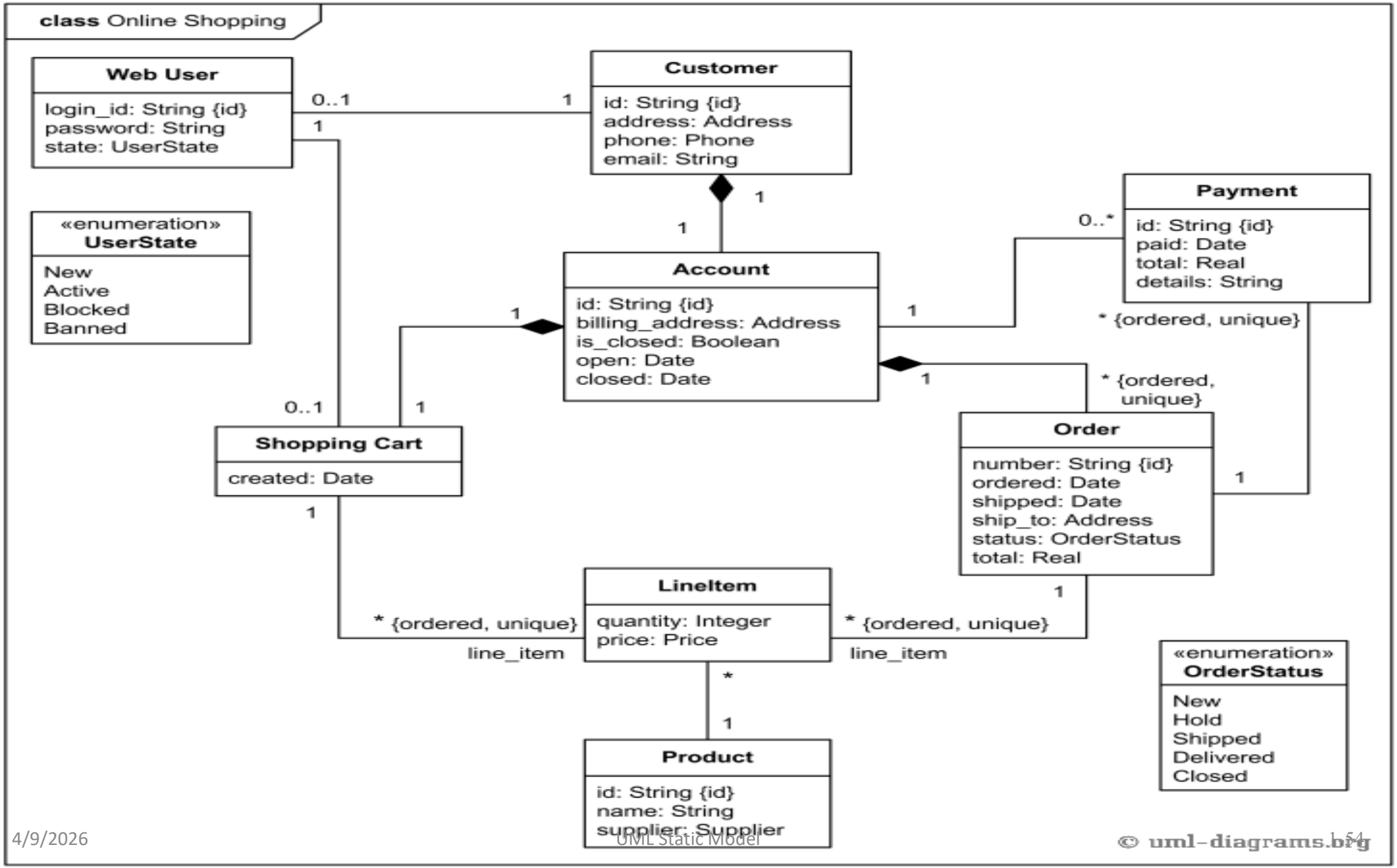
Association – Multiplicity[1]

- A teacher teaches 1 to 3 courses (subjects)
- Each course is taught by only one teacher.
- A student can take between 1 to 5 courses.
- A course can have 10 to 300 students.



Case Study: Online Shopping [8]

- Consider system for online shopping, there are Customer, Web User, Account, Shopping Cart, Product, Order, Payment, etc. It could be used as a common ground between business analysts and software developers.
- Each customer has unique id and is linked to exactly one **account**. Account owns shopping cart and orders. Customer could register as a web user to be able to buy items online. Customer is not required to be a web user because purchases could also be made by phone or by ordering from catalogues. Web user has login name which also serves as unique id. Web user could be in several states - new, active, temporary blocked, or banned, and be linked to a **shopping cart**. Shopping cart belongs to account.
- Account owns customer orders. Customer may have no orders. Customer orders are sorted and unique. Each order could refer to several **payments**, possibly none. Every payment has unique id and is related to exactly one account.
- Each order has current order status. Both order and shopping cart have **line items** linked to a specific product. Each line item is related to exactly one product. A product could be associated to many line items or no item at all.
- Source
  - https://www.uml-diagrams.org/examples/online-shopping-domain-uml-diagram-example.html


class Online Shopping
Web User
login_id: String {id}
password: String
state: UserState
0..1
1
Customer
id: String {id}
address: Address
phone: Phone
email: String
1
1
«enumeration»
UserState
New
Active
Blocked
Banned
Payment
id: String {id}
paid: Date
total: Real
details: String
0..*
Account
id: String {id}
billing_address: Address
is_closed: Boolean
open: Date
closed: Date
1
1
* {ordered, unique}
1
* {ordered, unique}
0..1
1
Shopping Cart
created: Date
1
Order
number: String {id}
ordered: Date
shipped: Date
ship_to: Address
status: OrderStatus
total: Real
1
1
LineItem
quantity: Integer
price: Price
* {ordered, unique}
line_item
* {ordered, unique}
line_item
*
1
Product
id: String {id}
name: String
supplier: Supplier
«enumeration»
OrderStatus
New
Hold
Shipped
Delivered
Closed
© uml-diagrams.org

Dynamic Model

- Dynamic model describes those aspects of the system that changes with the time.
- It is used to specify and implement control aspects of the system.
- It depicts states, transitions, events and actions
  - An event is a one-way transmission of information from one object to another
- The dynamic model includes
  - event trace diagrams describing scenarios
  - a scenario is a sequence of events that occurs during one particular execution of a system

Dynamic Model

- Each basic execution of the system can be represented by a scenario.
  - Scenario may include all events in the system or it may include only those events generated by certain objects.
  - The scope of scenario may vary
- The outcomes of dynamic modeling are scenario, event-trace diagram and state diagram.
- The dynamic model is represented graphically by sequence diagram and state diagram.
  - A state corresponds to the interval between two events received by an object and describes the "value" of the object for that time period.
- Event trace diagram is also known as interaction diagram

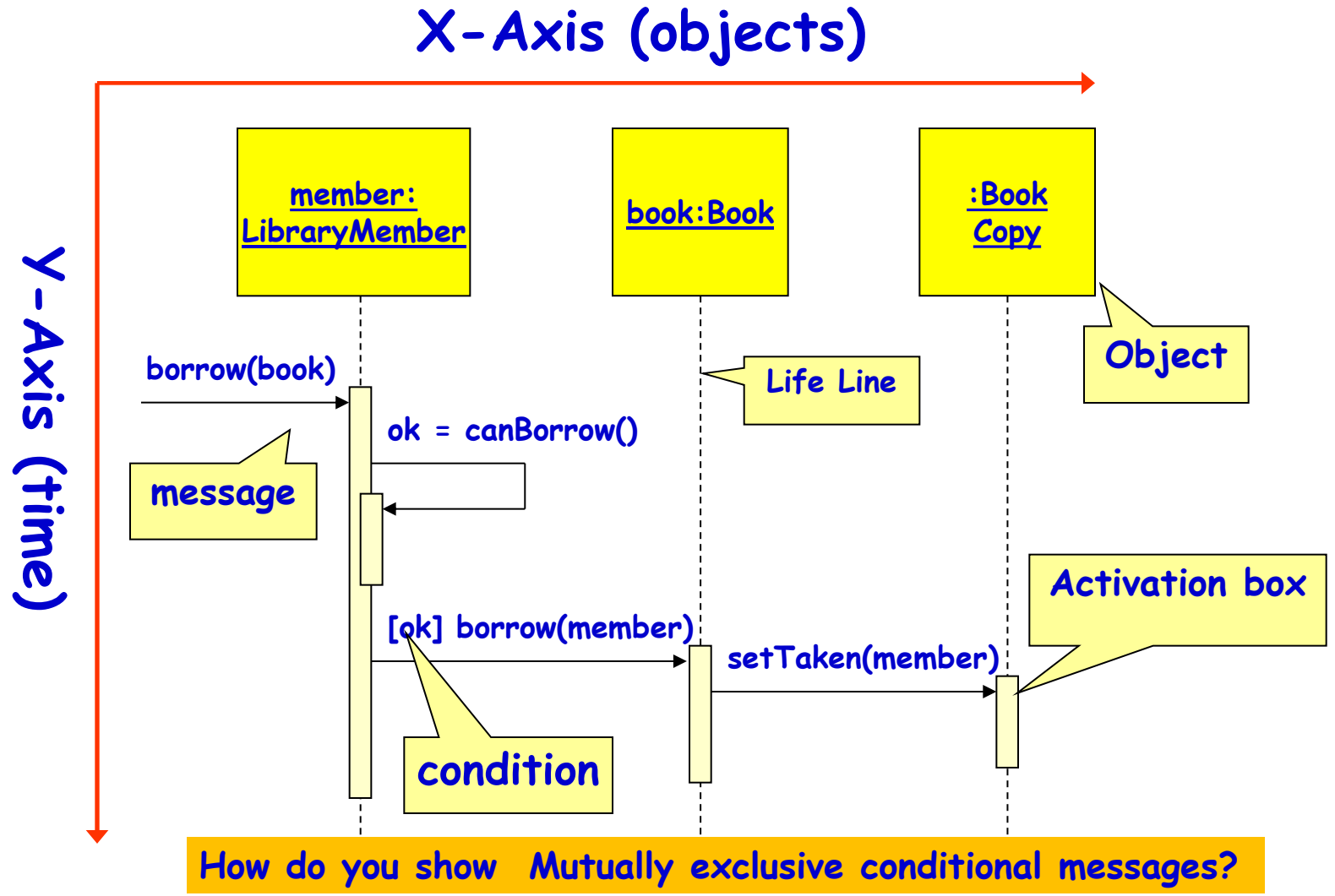
Interaction Diagrams [1]

- A series of diagrams describing the *dynamic behavior* of an object-oriented system.
  - A set of messages exchanged among a set of objects within a context to accomplish a purpose.
- Often used to model the way a use case is realized through a sequence of messages between objects.
- The interactions are represented by sequence diagram therefore sequence diagram is one type of interaction diagram.

Interaction Diagrams (Cont.) [1]

- The purpose of Interaction diagrams is to:
  - Model interactions between objects
  - Assist in understanding how a system (a use case) actually works
  - Verify that a use case description can be supported by the existing classes
  - Identify responsibilities/operations and assign them to classes
- Interaction Diagram
  - Sequence diagram
  - Collaboration diagram

Elements of A Sequence Diagram[1]



Elements of Sequence Diagram

- It consists of many parts
  - Dimension
    - Object Dimension
    - Time Dimension
  - Actor
  - Life-Line
  - Activation
  - Message
    - A message defines a particular communication between Lifelines of an interaction.
- Source
  - https://www.youtube.com/watch?v=_Mzi1rYtI5U

Messages

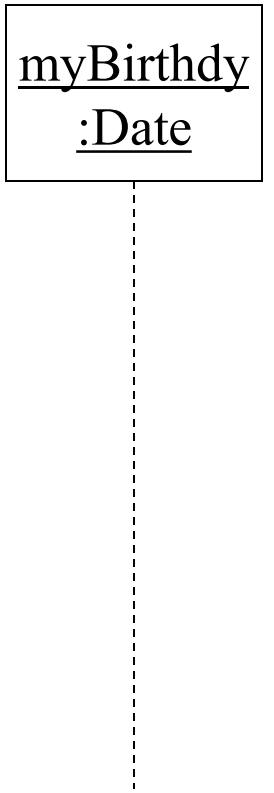
- Synchronous Message
  - Sender must wait for a response
- Asynchronous Message
  - It does not wait for a response
- Return Message
- Create Message
- Object Destruction
- Reflexive Message or Self Message
- Comment

Sequence Diagram (More Elements)

- Focus of Control
  - The focus of control indicates duration time of activation when processing is taking place within that object
- Control Object
  - Manages the collaboration of objects and has same name with use case
- Boundary Object
  - Manages the dialogue between the actor and the system
- Guards
  - To model conditions in sequence diagram
- Alt
  - It gives a choice between two or more message sequences.

Object

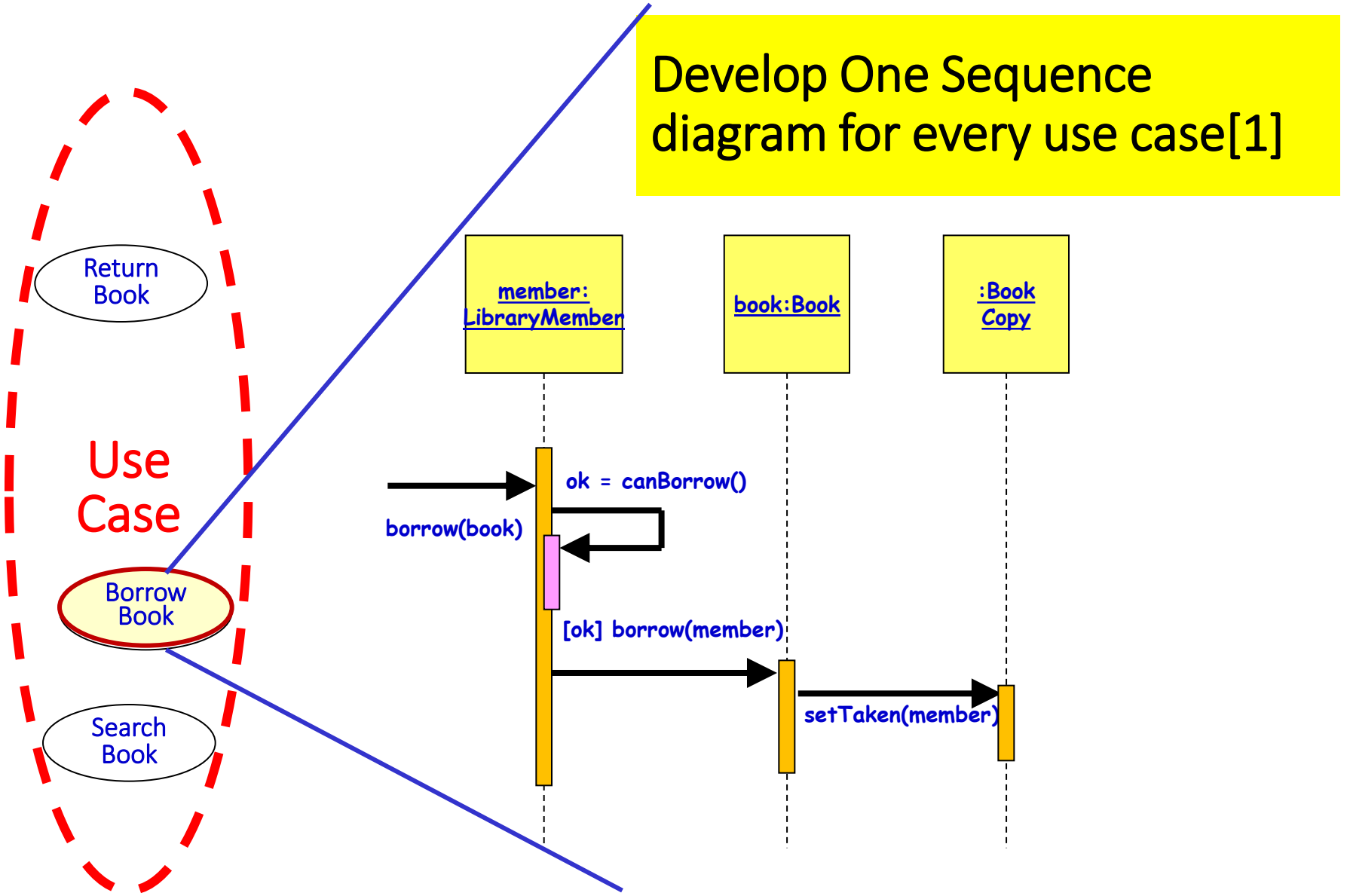
- Object Naming:
  - syntax: *[instanceName][:className]*
  - Name classes consistently with your class diagram (same classes).
  - Include instance names when objects are referred to in messages or when several objects of the same type exist in the diagram.
- The *Life-Line* represents the object's life during the interaction



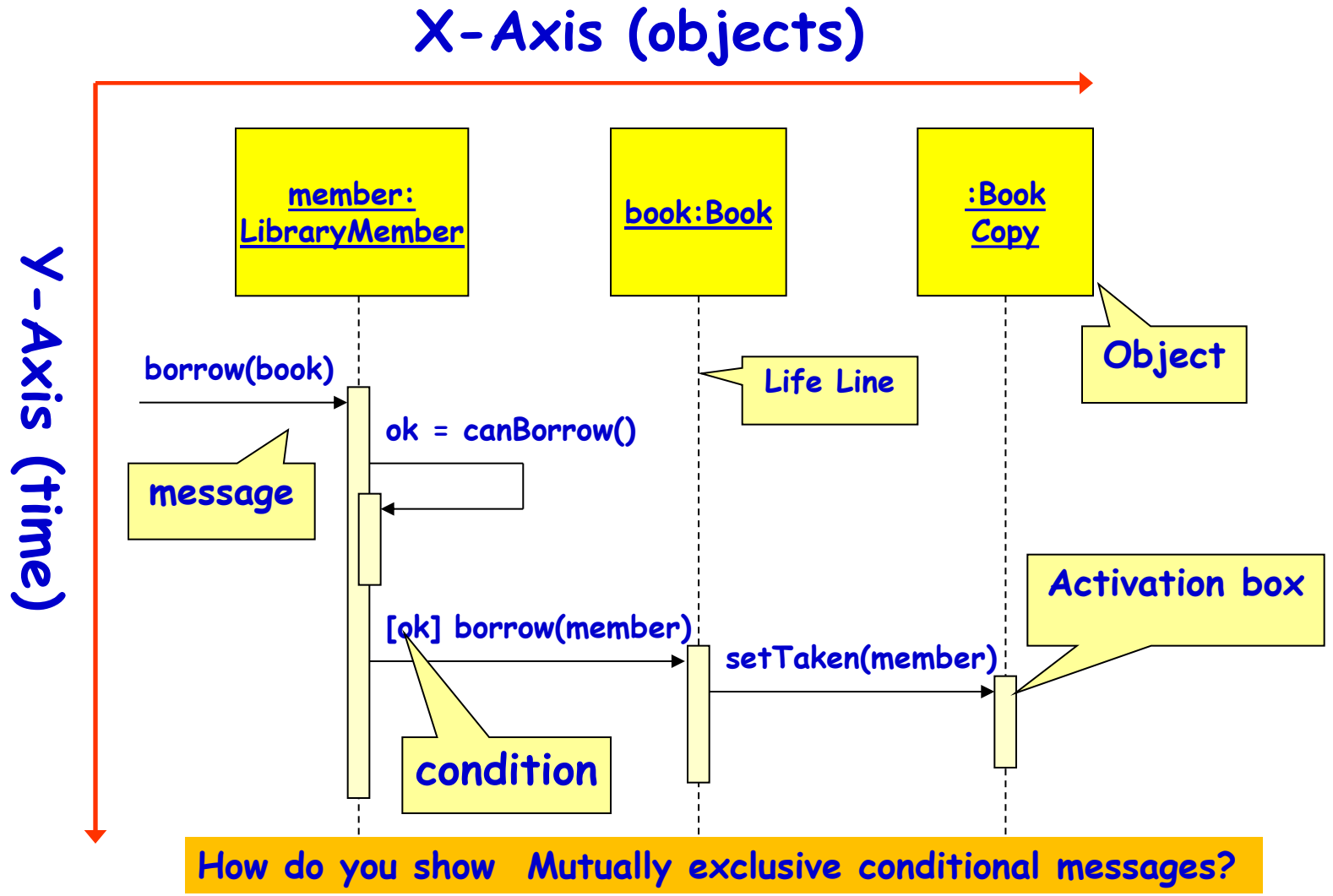
Messages (Cont.)

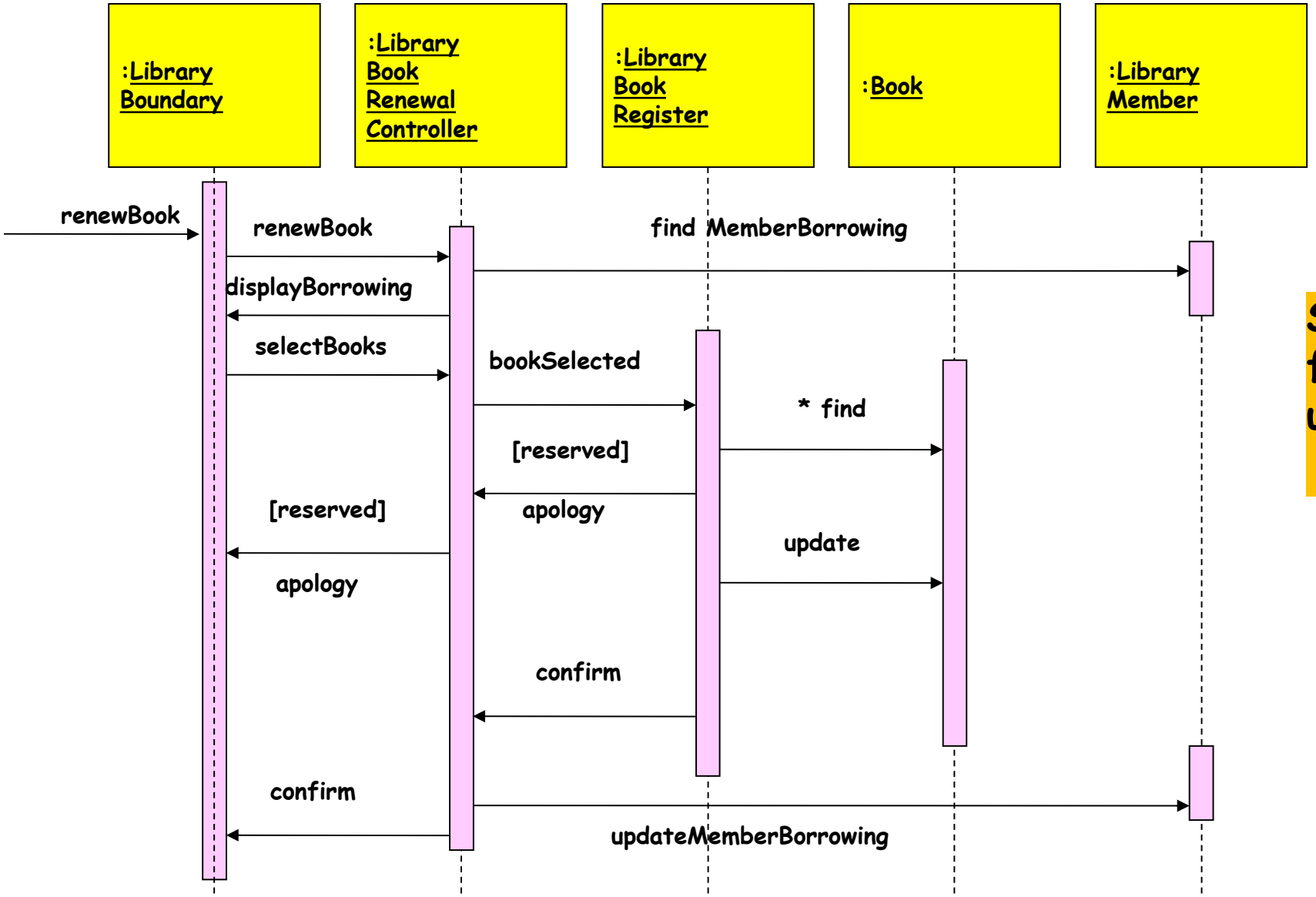
- A message is represented by an arrow between the life lines of two objects.
  - Self calls are also allowed
  - The time required by the receiver object to process the message is denoted by an *activation-box.*
- A message is labeled at minimum with the message name.
  - Arguments and control information (conditions, iteration) may be included.

Develop One Sequence diagram for every use case[1]



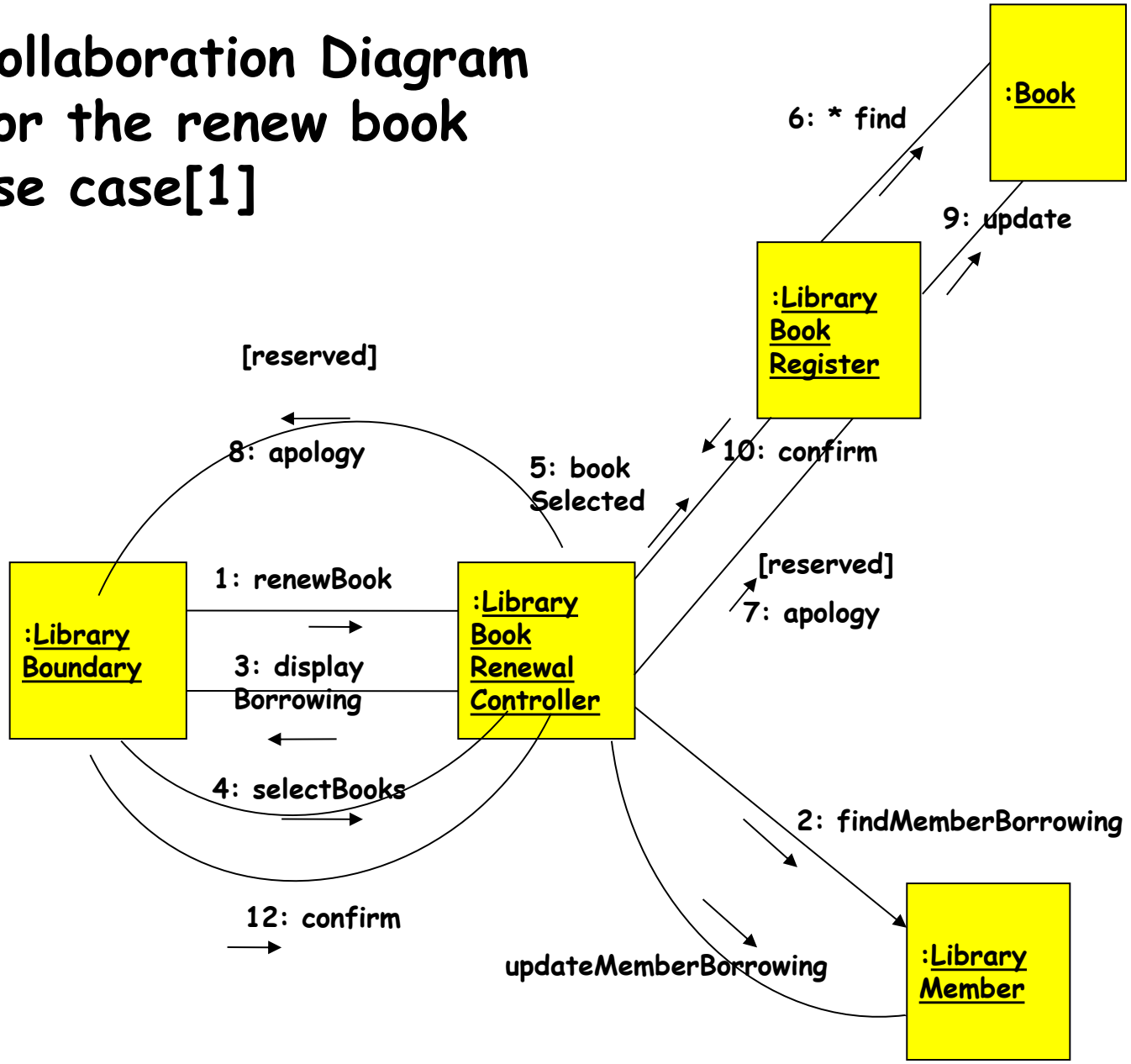
Elements of A Sequence Diagram[1]

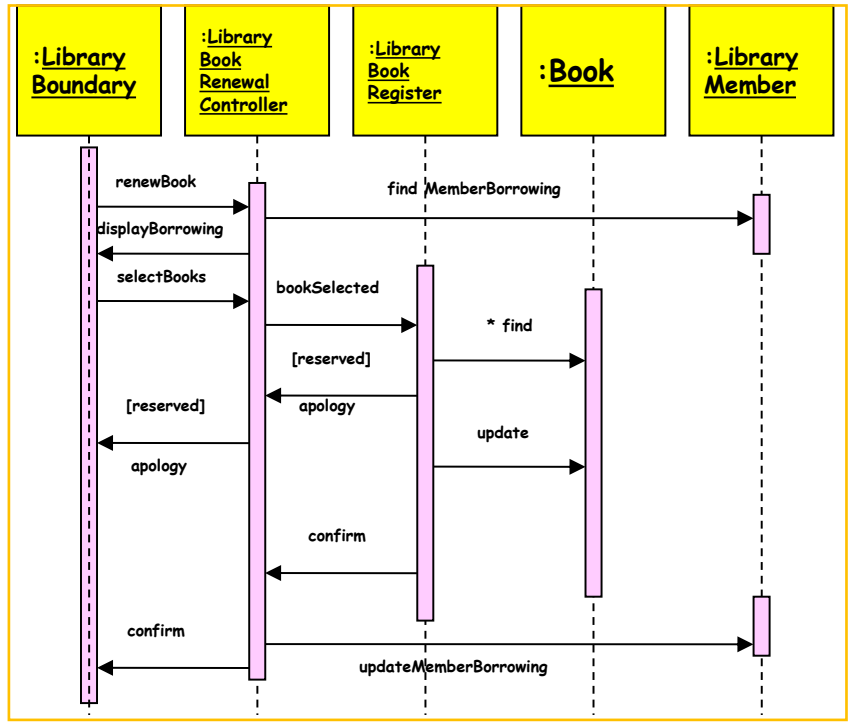




Sequence Diagram for the renew book use case[1]

Collaboration Diagram for the renew book use case[1]





References

- Herbert Schildt, Java 2: The Complete Reference, McGraw-Hill , 5th Edition, 2018.
- Cay S. Horstmann, Gary Cornell, *Core Java Volume I - Fundamentals*, Pearson Education , 11th Edition, 2021
- Grady Booch, Robert A. Maksimchuk, Michael W. Engle, *Object-Oriented Analysis and Design with Applications*, Addison-Wesley , 3rd Edition, 2007
- E. Balagurusamy, *Programming with Java: A Primer*, McGraw Hill , 6th Edition, 2019
- Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley , 1st Edition, 1994
- PPT available for the respective books

THANK YOU